then the -n option, which is being passed to Minitest and is the name of the individual test method you want to run. This is the test name after ActiveSupport::TestCase is done munging it, so what's in your file as test "a completed task is complete" is on the command line as test_a_completed_task_is_complete—with the word test prepended to the name and all the spaces turned into underscores.

If that seems like a lot of annoying typing, I can help a tiny bit. The minitest-line gem adds a different switch to the command line, -l,[1] which takes the line number of the test, changing your command line to this relatively brief one:

```
$ ruby -Ilib:test test/models/task_test.rb -l 5
```

The line number doesn't have to be the first line of the test. If you pick an intermediate line, it will run the test that contains that line.

Prescription 46	If you specify a test by line number, be sure not to add lines to the file. Added lines could make it so you aren't running the test you think you're running.

Cucumber

Cucumber's set of options is almost identical to RSpec's. Again, I recommend using the cucumber command line directly rather than going through rake. You can specify one or more files, append line numbers with a colon, and allow or deny tags with the -t or --tag option. (The main difference is that Cucumber tags start with an @, which you must include in the command line.)

Cucumber privileges one special tag, @wip, which is short for "work in progress." Cucumber scenarios tagged with @wip won't count as failures in a test run (much like RSpec's pending). Cucumber also has the separate -w or --wip, which runs Cucumber in reverse and fails if there are any passing scenarios.

Running Rails in the Background

One difference between TDD in a Rails context and TDD in SUnit's original Smalltalk environment is that testing a Rails program typically requires that Ruby be started from the command line and that the Rails environment be loaded. Starting from the command line can take quite a bit of time on even a moderate-size Rails project. If you're trying to run, say, a single file's worth of tests that you expect to take about a second, having the Rails startup take more than a minute can break your flow. In contrast, the Smalltalk tests reside inside the same live process as the code and can therefore start instantly.

1. https://github.com/judofyr/minitest-line